

PrimeNet Full Codebase Deep Dive

What this document is

This is a deep technical reference that explains:

- how the codebase is organized and curated
- what each major runtime file does
- what each script in `scripts/` does
- what each module route layer does
- data flow, DB touchpoints, dependencies, and operational caveats

1) Runtime Core (Application Backbone)

`app.py`

- ****Role****: Flask endpoint and composition root.
- ****Curates****:
 - app init and config
 - blueprint registration
 - DB init + migration startup
 - scheduler startup
 - global password-rotation policy
 - generic error handlers
- ****Execution order****:
 1. create app and set config
 2. import/register blueprints
 3. run migrations/init routines
 4. start scheduler (env controlled)
 5. enforce request-level auth/password policy
- ****Caveat****: scheduler and pull watcher behavior is controlled via env flags.

`database\_enhanced.py`

- ****Role****: app/admin data service layer over `ncm\_users.db`.
- ****Curates****:

- auth/session tables and lifecycle
- activity logging
- tasking and task updates
- profile/preferences
- config history and report archive
- config scheduler task tables
- ****Key logic****:
- user creation/login/password policy checks
- session token management
- role/permission checks for task actions
- compatibility upgrades (`ALTER TABLE` guarded calls)

`db/runtime.py`

- ****Role****: central SQLite runtime adapter.
- ****Curates****:
- query execution with lock retry
- WAL/busy-timeout connection tuning
- standard DB connectors
- cross-DB attach model for analytics
- ****Why it matters****: route modules can query metadata and PM DBs consistently while background writers run.

`sync\_config.py`

- ****Role****: central config registry.
- ****Curates****:
- absolute paths and DB locations
- SFTP server definitions
- ingestion/scheduler knobs
- retention and incremental/full modes
- helper maps for PM/metadata conventions

`ncm\_core.py`

- ****Role****: pure conversion/comparison engine used by web routes.
- ****Curates****:
- XML → Excel conversion
- Excel → XML conversion

- XML comparison and report building

2) Script-by-Script Deep Inventory (`scripts/`)

2.1 Orchestration Pipelines

`pull\_all\_raw.py`

- clears/stages raw folders
- orchestrates hourly pulls in sequence
- delegates parsing/loading to downstream scripts

`pull\_all\_raw\_daily.py`

- daily equivalent of the master pull orchestration
- targets daily raw folder paths and daily data cadence

`pull\_and\_load\_daily.py`

- full daily wrapper: pull then load
- intended for one command daily refresh operations

2.2 Vendor Pullers

`pull\_huawei\_raw.py`

- SFTP puller for Huawei PM/groups exports
- resolves latest files/subfolders, handles ZIP extraction
- outputs normalized raw files under Huawei raw folders

`pull\_nokia\_raw.py`

- same function as above for Nokia PM/groups per RAT folders
- includes candidate folder probing and latest-file rules

`pull\_huawei\_raw\_daily.py`* / *`pull\_nokia\_raw\_daily.py`

- daily-interval counterparts that write into daily raw folders

`pull\_metadata\_raw.py`

- pulls latest metadata snapshot exports from metadata SFTP source
- writes to metadata raw staging folders for loader consumption

2.3 Loaders and Data Shaping

``load_raw_csv_to_databases.py``

- main ingestion engine
- reads raw files (CSV/XLSX/ZIP-derived content)
- normalizes vendor schema differences
- loads/upserts PM/group/metadata tables
- supports incremental/time-filtered ingestion modes
- updates/refreshes metadata summary tables and related structures

``load_raw_daily_to_databases.py``

- wrapper around main loader for daily scope
- applies daily-specific flags and behavior

``build_kpi_headers_db.py``

- builds KPI header/catalog DB from live PM schemas/tables

2.4 Neighbor Flows

``pull_nokia_neighbor_raw.py``

- pulls Nokia neighbor exports to raw neighbor folders

``pull_huawei_neighbor_raw.py``

- pulls Huawei neighbor ZIP exports, extracts and classifies by RAT

``load_nokia_neighbor_raw_to_db.py``

- loads Nokia neighbor raw data into neighbor DB tables
- supports slim/wide handling patterns

``load_huawei_neighbor_wide_to_db.py``

- loads Huawei neighbor wide/raw structures into dedicated DB

``build_neighbor_kpi_db.py``

- migration/cleanup style script for neighbor schema evolution

``check_neighbor_dbs.py``

- diagnostic script: report raw/DB availability and row counts

2.5 Femto Flows

``pull_femto_raw.py``

- recursive pull of femto TGZ payloads to raw storage

``load_femto_pm_to_db.py``

- parses femto archives and loads femto PM DB tables
- applies retention cleanup policies

``parse_femto_metadata.py``

- parses and imports femto metadata/counter structures

``import_femto_catalogs.py``

- imports local femto KPI/counter catalogs into DB tables

2.6 Watcher / Scheduler-Adjunct

``watch_remote_new_files_and_pull.py``

- polls remote paths
- keeps signature state file
- triggers selective pull/load when new files appear
- supports one-shot and loop modes

2.7 Schema / Cleanup Utilities

``init_database.py``

- one-shot bootstrap: migrations + app DB initialization

``add_new_tables.py``

- idempotent creation of newer app/admin tables

``add_map_tables.py``

- creates map-related tables/indexes for legacy compatibility

``drop_legacy_performance_storage.py``

- destructive cleanup of legacy PM/group DB files
- confirmation-gated operation

``drop_duplicate_kpis.py``

- cleanup pass for duplicated KPI entries/rows

2.8 Diagnostics / Probes

``dump_db_headers.py``

- prints table/column headers for key databases

``audit_sqlite_databases.py``

- audits row counts, table health, and DB surface state

``debug_huawei_pm_zip.py``

- parser/debug helper for Huawei PM ZIP content behavior

``_inspect_huawei_now.py``

- quick status probe for Huawei PM DB state

``_reload_huawei_only.py``

- targeted Huawei-only reload sequence

``_probe_trend_flow.py``

- checks trend retrieval pipeline behavior end-to-end

``generate_infrastructure_pdf.py``

- local utility for converting markdown docs to PDF

2.9 Additional Integration Helpers

``import_local_files.py``

- local-file ingest helper into PM/metadata pipeline conventions

``load_neighbor_reports.py``

- materializes neighbor report data from raw/DB sources

``pull_performance_project_neighbor.py``

- project-specific neighbor pull orchestration utility

3) Blueprint Route Modules: What each one does

``modules/sync/routes.py``

- admin sync APIs: status/progress/history/trigger/test/inspect
- logs admin actions into sync history
- spawns background trigger threads for long operations

``modules/performance/routes.py``

- central KPI analytics API surface
- provides trend, group, report, filter, and PM table endpoints
- heavy dynamic SQL and schema-awareness logic

``modules/network_map/routes.py``

- map pages and APIs (sites, wedges, KPI popups, exports, neighbor overlays)
- combines metadata and neighbor datasets for geospatial analysis

``modules/conflict_map/routes.py``

- conflict-map endpoints and refresh/export actions
- delegates risk/math operations to ``modules/conflict_map/logic.py``

``modules/network_management/routes.py``

- operational inventory/conflict summary APIs for management view

``modules/reports/routes.py``

- report generation, archive, download, and type listing
- integrates metadata helpers and cached enrichment patterns

``modules/admin_panel/routes.py``

- user admin operations (roles/status/users)
- PM data freshness diagnostics for operations

``modules/config_history/routes.py``

- upload/list/version/diff/download/delete config versions

``modules/drive_test_viewer/routes.py``

- upload and parse drive-test artifacts (GPX/NMFS flows)

``modules/excel_generator/routes.py``

- Excel-to-XML workflow around ``ncm_core``

``modules/xml_parser/routes.py``

- XML-to-Excel workflow around ``ncm_core``

``modules/ne_comparison/routes.py``

- compare two XMLs and generate downloadable result output

``modules/parameter_dictionary/routes.py``

- list/search parameter dictionaries and MO metadata

``modules/task_scheduler/routes.py``

- task scheduler CRUD and file/result lifecycle endpoints

``modules/user_profile/routes.py``

- profile updates, password change, preferences, activity, photo request flow

``modules/femto_pm/routes.py``

- femto KPI catalog and trend endpoints

4) Key Helper Modules and Why They Exist

``modules/sync/scheduler.py``

- schedules and orchestrates recurring pipeline jobs
- controls job registration based on env flags
- handles progress state and sync logging

``modules/sync/db_migration.py``

- idempotent DB migration/creation for operational databases

``modules/sync/metadata_active_sql.py``

- shared SQL builders for per-tech metadata unions and activity status logic

``modules/network_map/neighbor_raw_linking.py``

- neighbor parsing/linking utilities and HO-related derivations

``modules/network_map/huawei_prs_tabular.py``

- parser for Huawei PRS tabular exports

``modules/conflict_map/logic.py``

- geospatial conflict scoring engine (distance/bearing/risk strictness)

``modules/reports/metadata_helpers.py``

- metadata SQL helper layer for report generation resilience

``modules/performance/kpi_catalog.py` / `kpi_mapping.py``

- KPI cataloging and mapping support for performance UI/API

``modules/sync/pm_processor.py``

- PM parser/normalization and import helpers

``modules/sync/sftp_client.py``

- reusable SFTP utility abstraction

``modules/sync/group_processor.py``

- group ingestion normalization logic

``modules/sync/metadata_processor.py``

- metadata ingest and consistency refresh helper logic

5) End-to-End Curation Model (How code is curated overall)

1. ``app.py`` composes runtime and guards policies.
2. Scheduler or admin trigger initiates pull/load workflows.
3. Pull scripts fetch raw artifacts into structured local folders.
4. Loader scripts normalize diverse vendor formats into canonical DB tables.
5. Module routes query, aggregate, and expose data to UI/API clients.
6. Admin/report/task/profile modules curate user workflow state in app DB.
7. Diagnostics scripts validate DB health and troubleshoot pipeline drift.

6) Critical Caveats and Modes

- SQLite is the canonical runtime backend.

- Several legacy paths are feature-flag gated or reset-disabled.
- Some scripts intentionally clear raw staging directories before pull.
- Destructive scripts exist and are confirmation-gated for safety.
- Data freshness depends on scheduler/watcher enablement or manual triggers.

7) Reading Order for Full Understanding

1. ``app.py``
2. ``routes/auth_routes.py``
3. ``database_enhanced.py``
4. ``db/runtime.py``
5. ``sync_config.py``
6. ``modules/sync/scheduler.py``
7. ``scripts/pull_all_raw.py`` + ``scripts/load_raw_csv_to_databases.py``
8. ``modules/performance/routes.py``
9. ``modules/network_map/routes.py``
10. remaining ``modules/*/routes.py``

This order provides a complete understanding from bootstrap to ingestion to API behavior.